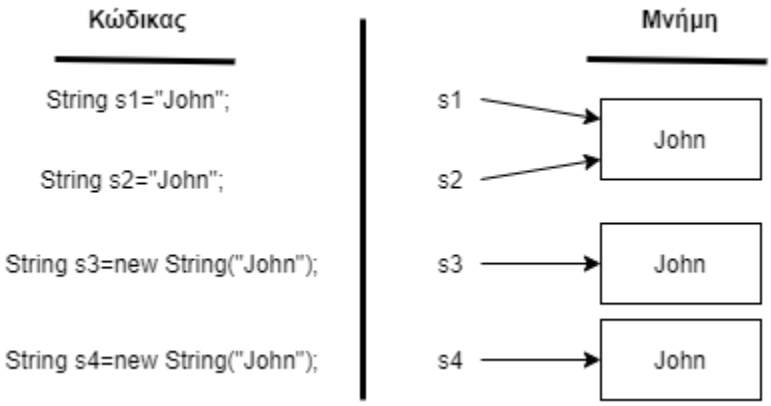


Σχήμα 3.4 *Δημιουργία μεταβλητών με την new*

Όπως δείχνει το σχήμα 3.4, οι μεταβλητές s1 και s2 δημιουργήθηκαν χωρίς χρήση της λέξης κλειδί new. Εφόσον έχουν την ίδια τιμή, κατανέμεται ο ίδιος χώρος στη μνήμη. Αντίθετα, οι μεταβλητές s3 και s4 δημιουργήθηκαν με χρήση της new. Σε αυτήν την περίπτωση κατανέμεται διαφορετικός χώρος στη μνήμη ακόμη και αν τα περιεχόμενα των μεταβλητών είναι ίδια. Επομένως, η s1 και η s2 περιέχουν την ίδια διεύθυνση μνήμης και άρα ισχύει s1==s2, δηλαδή οι μεταβλητές συγκρινόμενες με τον τελεστή ισότητας (ενότητα 4.1.4.1) θεωρούνται ίσες. Οι μεταβλητές s3 και s4 αναφέρονται στο ίδιο περιεχόμενο που είναι κατανεμημένο όμως σε διαφορετικές θέσεις στη μνήμη. Επομένως, στην s3 αποθηκεύεται διαφορετική διεύθυνση από ότι στην s4 και συνεπώς δεν ισχύει η ισότητα μεταξύ s3 και s4.
Αυτή η διαχείριση έχει ως στόχο τη βελτίωση της αποτελεσματικότητας του κώδικα και βασίζεται στο γεγονός ότι οι τιμές του τύπου String και των κλάσεων των θεμελιωδών τύπων είναι αμετάβλητες (immutable), δηλαδή από τη στιγμή που θα δημιουργηθεί μια τιμή ενός τέτοιου τύπου διασφαλίζεται πως δεν θα μεταβληθεί ως το τέλος του προγράμματος. Οι αμετάβλητοι τύποι εξυπηρετούν στον πολυνηματικό προγραμματισμό και χαρακτηρίζονται ως νηματικά ασφαλείς (thread safe).

3.8.1 Συλλέκτης απορριμμάτων

Εδώ όμως προκύπτει ένα άλλο θέμα. Το θέμα της απελευθέρωσης της μνήμης από τιμές μεταβλητών αναφοράς που δεν είναι χρήσιμες πλέον. Οι τοπικές μεταβλητές όπως είδαμε καταστρέφονται στο τέλος του μπλοκ στο οποίο δηλώθηκαν. Αυτό ισχύει είτε είναι μεταβλητές τιμής είτε μεταβλητές αναφοράς. Τι γίνεται όμως με τις τιμές των μεταβλητών αναφοράς που δεσμεύουν χώρο στη μνήμη δυναμικά; Είναι ένα θέμα που στην τυπική περίπτωση δεν απασχολεί ιδιαίτερα τον προγραμματιστή της Java καθώς η διαχείριση της δυναμικής μνήμης στην Java γίνεται από τον συλλέκτη απορριμμάτων (garbage collector). Ο συλλέκτης απορριμμάτων θεωρεί τις τιμές αναφοράς ως επιλέξιμες για αποδέσμευση μνήμης όταν δεν υπάρχει προς αυτές καμιά αναφορά μέσα στο πρόγραμμα.

```
static void garbageCollection() { //Κώδικας 3.4
    String s = new String("John");
    s = new String("Mary");
    System.out.println(s);
}
```

Κώδικας 3.4 Επιλέξιμες τιμές αναφοράς για αποδέσμευση μνήμης

Στην τελευταία γραμμή της garbageCollection στον κώδικα 3.4, το String John είναι ήδη επιλέξιμο προς αποδέσμευση. Η μόνη αναφορά στο John είναι η s έτσι όπως δημιουργείται στην πρώτη γραμμή. Στη δεύτερη γραμμή όμως, η αναφορά s ανακατευθύνεται σε άλλη τιμή. Έτσι το String John μένει χωρίς καμία αναφορά, δηλαδή είναι αδύνατο να το προσπελάσουμε μέσα από το πρόγραμμά μας καθώς το πρόγραμμά μας δεν γνωρίζει τη διεύθυνσή του. Με το τέλος της εκτέλεσης της garbageCollection, η τοπική μεταβλητή

s καταστρέφεται, οπότε μένει και το String Mary χωρίς αναφορά και άρα καθίσταται και αυτό επιλέξιμο προς αποδέσμευση.

Το πότε ακριβώς θα πραγματοποιηθεί η αποδέσμευση της μνήμης εξαρτάται από τον συλλέκτη απορριμμάτων ο οποίος έχει δικούς του αλγόριθμους με στόχο την αποδοτική διαχείριση της μνήμης.

3.9 Ασκήσεις

1. Περνάει με επιτυχία μεταγλώττιση η x1;

```
static void x1() {
     int k = 1, K = 2;
     System.out.println(k + " " + K);
}
```

Παραβιάζει κάποια από τις συμβάσεις ονοματολογίας; Εξηγήστε την απάντησή σας.

2. Περνάει με επιτυχία μεταγλώττιση η x2; Εξηγήστε την απάντησή σας.

```
static void x2() {
     int k = 1, j;
     j = j + 1;
     System.out.println(k + " " + j);
}
```

3. Περνάει με επιτυχία μεταγλώττιση η x3; Εξηγήστε την απάντησή σας.

```
static void x3() {
     int k = 1;
     final int j = k;
     j = j + 1;
     System.out.println(k + " " + j);
}
```

4. Περνάει με επιτυχία μεταγλώττιση η x4; Εξηγήστε την απάντησή σας.

```
static void x4() {
     int k = 1;
     {
          k = 2;
     }
     System.out.println(k);
}
```

5. Περνάει με επιτυχία μεταγλώττιση η x5; Εξηγήστε την απάντησή σας.

```
static void x5() {
     int k = 1;
     {
          int k = 2;
     }
     System.out.println(k);
}
```

6. Ποια είναι η έξοδος της x6;

```
static void x6() {
     int k = 010;
     System.out.println(k);
}
```

7. Ποια είναι η έξοδος της x7;

```
static void x7() {
     int k = 010, j = 10;
     System.out.println(k + j);
}
```

8. Ποια είναι η έξοδος της x8;

```
static void x8() {
     int k = 010, j = 0x10;
     System.out.println(k + j);
}
```

9. Ποια από τα παρακάτω ονόματα μεταβλητών παραβιάζουν τους κανόνες και ποια τις συμβάσεις ονοματολογίας;

```
int sum; char Start; final char END; boolean ExitLoop;
int finalsum; double 3oAthroisma; byte day; float float_10;
float _1_;
```

10. Υπάρχει διαφορά μεταξύ των κυριολεκτικών σταθερών 8 και 8.0;

Υποδείξεις: Προσπαθήστε να απαντήσετε χωρίς τη βοήθεια του μεταγλωττιστή. Χρησιμοποιήστε τον μεταγλωττιστή για να ελέγξετε τις απαντήσεις σας.

Βιβλιογραφία

[1] “Primitive Data Types (The Java™ Tutorials > Learning the Java Language > Language Basics).” https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html (accessed Sep. 14, 2021).

[2] “Java All-in-One For Dummies, 6th Edition | Wiley,” Wiley.com. https://www.wiley.com/en-bb/Java+All+in+One+For+Dummies%2C+6th+Edition-p-9781119680451 (accessed Sep. 14, 2021).

[3] “Unicode 13.0.0.” https://unicode.org/versions/Unicode13.0.0/ (accessed Sep. 14, 2021).

[4] “The Java® Language Specification, Chapter 5. Conversions and Contexts, §5.1.2, §5.1.3, §5.1.4.” https://docs.oracle.com/javase/specs/jls/se10/html/index.html (accessed Sep. 14, 2021).

Κεφάλαιο 4

4 Τελεστές και βασική Είσοδος και Έξοδος

Η Java ενσωματώνει μια σειρά από συναρτήσεις που χρησιμεύουν ώστε να μπορούμε να κάνουμε διάφορες πράξεις (operations). Κάθε πράξη περιλαμβάνει τον τελεστή (operator) και τους τελεστέους (operands). Για παράδειγμα, στην πράξη 5+3, τελεστής είναι το σύμβολο της πρόσθεσης, + και τελεστέοι τα 5 και 3. Προκειμένου να διευκολυνθεί η έκφραση στο πρόγραμμα τέτοιων πράξεων, η Java παρέχει ειδικό συντακτικό κλήσης των τελεστών έτσι ώστε οι ανάλογες εκφράσεις να μην ακολουθούν το τυπικό συντακτικό κλήσης συναρτήσεων αλλά να διατυπώνονται κατά τρόπο που ομοιάζει με την εκτεταμένη χρήση τους στα μαθηματικά και άλλους τομείς.

Ένα άλλο ζήτημα, αναγκαίο για τα προγράμματά μας, είναι η βασική είσοδος και έξοδος. Πρόκειται για τις διαδικασίες που επιτρέπουν την επικοινωνία και αλληλεπίδραση μεταξύ χρήστη και προγράμματος. Πώς δηλαδή θα πάρω δεδομένα από τον χρήστη και πώς θα παρουσιάσω σε αυτόν τις πληροφορίες που θέλω;

Σε αυτό το κεφάλαιο παρουσιάζουμε τους βασικούς τελεστές που υποστηρίζει η Java [1] ταξινομώντας τους σε κατηγορίες ανάλογα με τα χαρακτηριστικά τους καθώς και τις βασικές λειτουργίες εισόδου από το πληκτρολόγιο και εξόδου στην οθόνη.

4.1 Τελεστές

Οι τελεστές (operators) στην Java οργανώνονται ανάλογα με τον αριθμό τελεστέων (operands) που δέχονται. Έτσι, έχουμε τους μοναδιαίους (unary) τελεστές, δηλαδή αυτούς που δέχονται έναν τελεστέο, τους δυαδικούς (binary), δηλαδή αυτούς με δύο τελεστέους και έναν τριαδικό (ternary) τελεστή. Επίσης, οι τελεστές κατηγοριοποιούνται σε αριθμητικούς (arithmetic), υποθετικούς (conditional) ή λογικούς (logical), σχεσιακούς (relational), τελεστές ψηφίου (bitwise) και τους τελεστές εκχώρησης (assignment).
Σε μια αλγεβρική έκφραση είναι δυνατό να συνδυάζονται περισσότεροι από έναν τελεστές. Για παράδειγμα, στην αλγεβρική έκφραση 2+3x5, συνδυάζονται οι τελεστές πρόσθεσης και πολλαπλασιασμού. Όπως γνωρίζουμε από την άλγεβρα, ο πολλαπλασιασμός έχει προτεραιότητα σε σχέση με την πρόσθεση. Επομένως, 2+3x5=17. Παρόμοια και στις εκφράσεις (expressions) της Java είναι δυνατό να συνδυάζονται πολλοί τελεστές. Επομένως, για τον συνεπή υπολογισμό των εκφράσεων έχουν καθοριστεί προτεραιότητες

και για τους τελεστές της Java. Στον πίνακα 4.1 παρουσιάζονται οι τελεστές διατεταγμένοι από υψηλότερη σε χαμηλότερη προτεραιότητα. Τελεστές στην ίδια γραμμή του πίνακα 4.1 έχουν ίση προτεραιότητα.

Τύπος	**Λειτουργία**	**Προτεραιότητα**
Μοναδιαίος	Μεταθεματική αύξηση/μείωση	++, --
	Προθεματική αύξηση/μείωση, συμπλήρωμα, λογικό NOT	++, --, ~, !
Αριθμητικοί	Πολλαπλασιασμός	*, /, %
	Πρόσθεση	+, -
Διολίσθησης	Διολίσθηση	<<, >>, >>>
Σχεσιακοί	Σύγκριση	<, >, <=, >=, instanceof
	Έλεγχος ισότητας	==, !=
Ψηφίου	AND ψηφίο προς ψηφίο	&
	XOR ψηφίο προς ψηφίο	^
	OR ψηφίο προς ψηφίο	|
Λογικοί	Λογικό AND	&&
	Λογικό OR	||
Τριαδικός		? :
Εκχώρησης	Εκχώρησης	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=

Πίνακας 4.1*: Προτεραιότητα τελεστών*

Ας υποθέσουμε την αλγεβρική έκφραση x=5+3-2. Η πρόσθεση και η αφαίρεση έχουν ίδια προτεραιότητα μεταξύ τους και ψηλότερη από την εκχώρηση. Επομένως, πρώτα θα υπολογιστεί η έκφραση 5+3-2 και το αποτέλεσμά της θα εκχωρηθεί στη μεταβλητή x. Πώς όμως θα αξιολογηθεί το τμήμα 5+3-2; Ο κανόνας εδώ είναι πως η αξιολόγηση γίνεται από αριστερά προς τα δεξιά, δηλαδή γίνεται πρώτα η πρόσθεση και το αποτέλεσμά της συνιστά τελεστέο για την αφαίρεση. Το ίδιο ισχύει για τις περισσότερες εκφράσεις της Java. Οι υπολογισμοί γίνονται από αριστερά προς τα δεξιά για όλους τους τελεστές πλην της εκχώρησης όπου η αξιολόγηση γίνεται από δεξιά προς τα αριστερά.

4.1.1 Ο τελεστής εκχώρησης

Η εκχώρηση είναι δυαδική πράξη και ο βασικός τελεστής της συμβολίζεται με τον χαρακτήρα '='. Στη γραμμή 2 του κώδικα 4.1 δηλώνονται τρεις ακέραιες μεταβλητές και αρχικοποιούνται οι δύο από αυτές. Στη γραμμή 3, εκχωρείται η τιμή 8 στη μεταβλητή i. Συνηθίζεται η πρώτη εκχώρηση σε μεταβλητή να αποκαλείται αρχικοποίηση (initialization). Η έκφραση (expression) στη γραμμή 3, επομένως, μπορούμε να πούμε πως αρχικοποιεί την i. Η αξιολόγηση γίνεται από τη δεξιά προς την αριστερή πλευρά της έκφρασης. Αυτό σημαίνει πως πρώτα θα υπολογιστεί ότι υπάρχει στα δεξιά του τελεστή εκχώρησης και στη συνέχεια η τιμή που θα προκύψει θα εκχωρηθεί στην i. Βέβαια, στη συγκεκριμένη περίπτωση, δεν απαιτούνται ιδιαίτεροι υπολογισμοί δεδομένου ότι στα δεξιά του τελεστή υπάρχει μια αριθμητική σταθερά. Θα μπορούσε όμως να υπάρχει και μια έκφραση που το αποτέλεσμα της αξιολόγησής της θα ήταν ακέραιος. Στη συνέχεια αυτής της ενότητας θα δούμε αρκετά τέτοια παραδείγματα.

```
1      static void assignmentDemo() { //Κώδικας 4.1
2          int i, j = 5, k = 9;
3          i = 8;
4          System.out.println(j = i);
5          j = k = i;
6          System.out.println(i + " " + j + " " + k);
7      }
```

***Κώδικας 4.1**: Ο τελεστής εκχώρησης*

Εκείνο όμως που έχει ιδιαίτερο ενδιαφέρον είναι πως η εκχώρηση επιστρέφει στο περιβάλλον κλήσης της μια τιμή και συγκεκριμένα την τιμή της i μετά την εκχώρηση. Αυτή η τιμή δεν αξιοποιείται στη γραμμή 3. Αξιοποιείται όμως στη γραμμή 4 που τυπώνει την έκφραση j=i. Αν τρέξετε τον κώδικα 4.1, θα διαπιστώσετε πως η τιμή που τυπώνεται είναι 8. Η εκχώρηση, αφού αξιολόγησε την τιμή του i, την εκχώρησε στην j και επέστρεψε στην println την τιμή του j μετά την εκχώρηση. Σε αυτό το χαρακτηριστικό βασίζεται και η υποστήριξη της πολλαπλής εκχώρησης (γραμμή 5).

Όπως φαίνεται στον πίνακα 4.1, είναι διαθέσιμοι και άλλοι τελεστές εκχώρησης. Η παρουσίασή τους γίνεται πιο κάτω σε αυτήν την ενότητα.

4.1.2 Αριθμητικοί τελεστές

Η Java παρέχει τους δυαδικούς τελεστές πρόσθεση, αφαίρεση, πολλαπλασιασμό, διαίρεση, υπόλοιπο διαίρεσης. Όλοι αυτοί οι τελεστές είναι δυαδικοί.

4.1.2.1 Πρόσθεση

Ο τελεστής πρόσθεσης συμβολίζεται με τον χαρακτήρα '+'. Στον κώδικα 4.2, προσθέτουμε το i με το j. Το αποτέλεσμα στέλνεται στην έξοδο όπου και τυπώνεται η τιμή 5. Στη γραμμή 4 όμως, το αποτέλεσμα ίσως φανεί λίγο παράδοξο. Πράγματι, η έξοδος του κώδικα της γραμμής 4 είναι i+j=32. Ο χαρακτήρας '+' χρησιμοποιείται τόσο για την πρόσθεση μεταξύ αριθμών, όσο και για τη σύνδεση String (concatenation) μεταξύ τους. Το αν θα ληφθεί ως πρόσθεση αριθμών ή ως σύνδεση String εξαρτάται από τους τελεστέους που συντάσσονται μαζί του. Αν έστω και ένας από τους τελεστέους είναι String, τότε εκτελείται η σύνδεση Strings. Στη γραμμή 4, η αξιολόγηση γίνεται από αριστερά προς τα δεξιά. Έτσι, στην έκφραση "i+j="+i εντοπίζεται πρώτα το String "i+j=", οπότε λαμβάνεται η απόφαση πως πρόκειται για σύνδεση. Έτσι ο επόμενος ακέραιος τελεστέος μετατρέπεται αυτόματα σε String, έτσι το αποτέλεσμα "i+j="+i είναι "i+j=3" και ο τύπος του είναι String. Στη συνέχεια μένει να συνδεθεί αυτό το String με την ακέραιη μεταβλητή j. Με παρόμοια διαδικασία, το τελικό αποτέλεσμα είναι "i+j=32".

```
 1      static void additionDemo() { //Κώδικας 4.2
 2          int i = 3, j = 2;
 3          System.out.println(i + j);
 4          System.out.println("i+j= " + i + j); //concatenation
 5          System.out.println("i+j= " + (i + j));
 6          String h = "hello", w = "World";
 7          System.out.println(h + " " + w);
 8          double k = 1;
 9          float m = 1f;
10          k = k + m;
11          char a = 'A';
12          System.out.println(a + 1);
13          System.out.println((char) (a + 1));
14      }
```

***Κώδικας 4.2**: Ο τελεστής πρόσθεσης*

Αν εκείνο που θέλουμε να πετύχουμε είναι να προστεθούν πρώτα το i με το j και το αποτέλεσμα να συνδεθεί με το "i+j= " ώστε να εκτυπωθούν μαζί, τότε θα πρέπει να δώσουμε προτεραιότητα στην πρόσθεση των ακέραιων, όπως δείχνει η γραμμή 5. Ισχύει και εδώ, όπως και στην άλγεβρα πως οι εκφράσεις μέσα σε παρενθέσεις αξιολογούνται κατά προτεραιότητα. Στη συνέχεια, οι γραμμές 22 και 23 παρουσιάζουν μια περίπτωση σύνδεσης String. Οι γραμμές, 8, 9 και 10 επιδεικνύουν πως ο τελεστής πρόσθεσης συντάσσεται με όλους τους αριθμητικούς τύπους. Ωστόσο, θα πρέπει να έχουμε υπόψη πως όταν συντάσσεται με δύο διαφορετικούς αριθμητικούς τύπους, ο τύπος του αποτελέσματος αποκτάται από τον τελεστέο που έχει το μεγαλύτερο μέγεθος. Στο παράδειγμά μας, η έκφραση k+m επιστρέφει double. Αν επιχειρούσαμε να

εκχωρήσουμε αυτό το αποτέλεσμα σε μεταβλητή τύπου float, ο μεταγλωττιστής θα παρήγαγε λάθος. Όπως έχουμε αναφέρει στην ενότητα 3.2, το αποτέλεσμα των εκφράσεων με byte ή short είναι αυτόματα int.
Ο τελεστής της πρόσθεσης δέχεται και τελεστέους τύπου char. Στην περίπτωση αυτή προστίθεται ο κωδικός των χαρακτήρων και το αποτέλεσμα της πράξης είναι τύπου int. Έτσι η γραμμή 12 θα προσθέσει στο 65 που είναι ο κωδικός του 'A' το 1 και θα τυπώσει το 66 που είναι ο κωδικός του 'B'. Μπορούμε να δούμε τον χαρακτήρα 'B' αν μετατρέψουμε τον τύπο του αποτελέσματος (type casting) όπως δείχνει η γραμμή 13.

4.1.2.2 Αφαίρεση

Ο τελεστής αφαίρεσης συμβολίζεται με τον χαρακτήρα '-'. Η γραμμή 3 του κώδικα 4.3 παράγει λάθος μεταγλώττισης εφόσον αποσχολιαστεί. Πράγματι, ανάλογα με όσα είπαμε στην περιγραφή του τελεστή πρόσθεσης, η έκφραση "i-j="+i επιστρέφει String. Στη συνέχεια η αφαίρεση ενός ακεραίου από ένα String δεν ορίζεται. Επομένως, σε αυτήν την περίπτωση, είμαστε αναγκασμένοι να δώσουμε προτεραιότητα στην αφαίρεση όπως φαίνεται στη γραμμή 4. Η αφαίρεση συντάσσεται και με δεδομένα τύπου char. Η συμπεριφορά της σε αυτήν την περίπτωση είναι ανάλογη με την πρόσθεση. Επίσης, ανάλογη με την πρόσθεση συμπεριφορά παρουσιάζει η αφαίρεση στην περίπτωση τελεστέων διαφορετικών αριθμητικών τύπων.

```
1    static void subtractionDemo() { //Κώδικας 4.3
2        int i = 3, j = 2;
3        //System.out.println("i-j= "+i-j);
4        System.out.println("i-j= " + (i - j));
5        char bita = 'B';
6        System.out.println(bita - 1);
7    }
```

***Κώδικας 4.3**: Ο τελεστής αφαίρεσης*

4.1.2.3 Πολλαπλασιασμός

Ο τελεστής του πολλαπλασιασμού συμβολίζεται με τον χαρακτήρα αστερίσκο, '*'. Το αποτέλεσμα της γραμμής 4 του κώδικα 4.4 είναι το αναμενόμενο. Αυτό οφείλεται στο γεγονός ότι ο πολλαπλασιασμός έχει προτεραιότητα έναντι της πρόσθεσης. Επομένως, στη γραμμή 4 υπολογίζεται πρώτα το γινόμενο i*j και στη συνέχεια συνδέεται το String "i*j=" με το αποτέλεσμα του πολλαπλασιασμού. Κατά συνέπεια οι παρενθέσεις που περικλείουν το γινόμενο i*j, στη γραμμή 5, είναι περιττές χωρίς να οδηγούν σε λάθος. Ο πολλαπλασιασμός δέχεται και τελεστέους τύπου char. Το αποτέλεσμα είναι το γινόμενο των κωδικών των χαρακτήρων. Έτσι, η γραμμή 6 θα τυπώσει 132, δηλαδή τον κωδικό του B επί 2.

```
1    static void multiplicationDemo() { //Κώδικας 4.4
2       int i = 3, j = 2;
3       char bita = 'B';
4       System.out.println("i*j= " + i * j);
5       System.out.println("i*j= " + (i * j));
6       System.out.println("bita*2=" + bita * 2);
7    }
```

***Κώδικας 4.4**: Ο τελεστής του πολλαπλασιασμού*

4.1.2.4 Διαίρεση

Ο τελεστής της διαίρεσης συμβολίζεται με τον χαρακτήρα πλάγια κάθετος, '/'. Ό,τι αναφέρθηκε για τον πολλαπλασιασμό, ισχύει κατ' αντιστοιχία και για τη διαίρεση. Επιπλέον, η διαίρεση διακρίνεται σε διαίρεση μεταξύ πραγματικών και διαίρεση μεταξύ ακέραιων. Αν οι τελεστέοι είναι ακέραιοι, τότε έχουμε ακέραια διαίρεση. Αν ένας τουλάχιστον από τους τελεστέους είναι πραγματικός, τότε έχουμε διαίρεση.

```
1    static void divisionDemo() { //Κώδικας 4.5
```

```
2         int i = 3;
3         int j = 2;
4         System.out.println("i/j= " + i / j);
5         System.out.println("i/j= " + (i / j));
6         System.out.println();
7         double d = 3, b = 2;
8         System.out.println("d/b= " + (d / b));
9         System.out.println("(double)i/j= " + (double) i / j);
10    }
```

***Κώδικας 4.5**: Ο τελεστής της διαίρεσης*

Η έξοδος της γραμμής 4 του κώδικα 4.5 είναι i/j=1. Και οι δύο τελεστέοι είναι ακέραιοι, οπότε η Java εκτελεί διαίρεση ακέραιων. Αντίθετα στη γραμμή 8, η έξοδος είναι d/b=1.5. Αν επιθυμούμε να κάνουμε διαίρεση πραγματικών μεταξύ ακέραιων μεταβλητών, θα πρέπει να μετατρέψουμε τον τύπο τουλάχιστον σε έναν από τους τελεστέους όπως φαίνεται στη γραμμή 9.

4.1.2.5 Υπόλοιπο διαίρεσης

Το υπόλοιπο συμβολίζεται με τον χαρακτήρα επί τοις εκατό, '%'. Εδώ έχουμε διαφοροποίηση μεταξύ υπολοίπου που προκύπτει από διαίρεση πραγματικών και υπολοίπου διαίρεσης ακέραιων.

```
1     static void moduloDemo() { //Κώδικας 4.6
2         int i = 3, j = 2; //1
3         System.out.println("i%j= " + i % j);
4         System.out.println("i%j= " + (i % j));
5         double d = 7.5, b = 3.5;
6         System.out.println("d%b= " + d % b);
7         System.out.println("d%b= " + (d % b));
8     }
```

***Κώδικας 4.6**: Το υπόλοιπο διαίρεσης*

Η έξοδος της γραμμής 3 του κώδικα 4.6 είναι i%j=1. Ο τύπος της έκφρασης i%j είναι int. Στη γραμμή 6 εκτελείται διαίρεση μεταξύ πραγματικών και η έξοδος είναι d%b=0.5. Ο τύπος της έκφρασης d%b είναι double.

4.1.3 Μοναδιαίοι τελεστές

Οι μοναδιαίοι τελεστές που υποστηρίζονται είναι το θετικό πρόσημο, το αρνητικό πρόσημο, οι τελεστές προσαύξησης, οι τελεστές μείωσης και ο λογικός τελεστής αντιστροφής.

4.1.3.1 Οι τελεστές προσήμου

Ο μοναδιαίος τελεστής μείον (unary minus operator) συμβολίζεται με τον χαρακτήρα '-' και επιστρέφει τον αντίθετο του τελεστέου. Στη γραμμή 5, ο κώδικας 4.7, τυπώνει -1. Ο τελεστής μείον δέχεται ως τελεστέους και πραγματικούς αλλά και χαρακτήρες. Ανάλογη είναι η λειτουργία του μοναδιαίου τελεστή συν (unary plus operator), '+'. Ωστόσο, ο τελεστής συν, ισχύει εξ ορισμού και έτσι δεν αναφέρεται σχεδόν ποτέ σε κώδικες Java.

```
1     static void unaryOps() { //Κώδικας 4.7
2         int i = 1;
3         System.out.println(i);
4         System.out.println(+i);
5         System.out.println(-i);
6         System.out.println();
7
8         System.out.println("Postfix increment");
9         System.out.println(i++);
```